Had I not converted the last line, the error would be more like this:

```
-[#<Task id: 980190963, project_id: nil, title: "Completed",
size: nil, completed_at: "2013-12-14 21:47:22",
created_at: "2013-12-15 21:47:22",
updated_at: "2013-12-15 21:47:22">]
+[]
```

I submit the first error message makes it easier to determine what's going on.

Another option, using recent RSpec parlance, is to use a *compound matcher*:

```
it "finds completed tasks" do
  complete = Task.create(completed_at: 1.day.ago, title: "Completed")
  incomplete = Task.create(completed_at: nil, title: "Not Completed")
  expect(Task.complete).to match(
    [an_object_having_attributes(title: "Completed")])
end
```

In this case, the `match` matcher expects the structure of `Task.complete` to be a one-element array where that one element has a `title` attribute with the value "completed." This style can be very readable and allows RSpec to create very useful error messages when it fails. Whether you use this style is a matter of preference. I find it quite readable in many cases; sometimes it's more clear to write the expectation using multiple matchers.

Once you've written your initial two-object test, write another test only if you can think of another pair of objects that would fail given the current code, such as if your finder had compound logic. If you were writing a method to find tasks that were both "completed" and "large," you might start with a test that has one object with both of those criteria and one with neither, and then write a second test that has an object with both criteria and an object that has only one of the two criteria.

I'm trying to avoid a combinatorial explosion where you create 16 objects to test a finder with four elements. Going two at a time and creating new pairs only if there's still a potential failure keeps each test small and easy to understand.

If you're testing for sort logic, you have to work around the fact that the order in which you add data to the database in setup is the order in which you get the objects back when you don't explicitly specify an order (because the default order is ascending by ID). If you input your data in the same order as the expected output, then the test passes before you do anything. That is bad.

If I need to test sort behavior, I create three objects: expected middle value first, expected high value second, and expected low value third. This pattern